

Tutorial: Sign-in users to a Python Flask web app by using Microsoft identity platform

Applies to:  Workforce tenants  External tenants ([learn more](#))

This tutorial guides you on securing a Python Flask Web app.

In this tutorial, you:

-  Create a Python Flask project
-  Install the required dependencies
-  Configure your Flask web app to use Microsoft identity platform for authentication
-  Test the sign-in and sign-out experience in your Flask web app

Prerequisites

Workforce tenant

- A workforce tenant. You can use your [Default Directory](#) or set up a new tenant.
- Register a new app in the [Microsoft Entra admin center](#) , configured for *Accounts in this organizational directory only*. Refer to [Register an application](#) for more details. Record the following values from the application **Overview** page for later use:
 - Application (client) ID
 - Directory (tenant) ID
- Add a client secret to your app registration. **Do not** use client secrets in production apps. Use certificates or federated credentials instead. For more information, see [add credentials to your application](#).

- [Python 3+](#) .
- [Visual Studio Code](#) or another code editor.

Create a Flask project

1. Create a folder to host your Flask application, such as *flask-web-app*.
2. Open a console window, and change to the directory to your Flask web app folder using the command

Bash

```
cd flask-web-app
```

3. Set up virtual environment

Depending on your operating system, run the following commands to set up your virtual environment and activate it:

For windows operating system:

Bash

```
py -m venv .venv  
.venv\scripts\activate
```

For macOS or Linux operating system:

Bash

```
python3 -m venv .venv  
source .venv/bin/activate
```

Install app dependencies

To install app dependencies, run the following commands:

Console

```
pip install flask  
pip install python-dotenv  
pip install requests  
pip install "ms_identity_python[flask] @ git+https://github.com/azure-samples/ms-identity-python@0.9"
```

The *ms_identity_python* library that you install automatically installs Microsoft Authentication Library (MSAL) for Python as its dependency. MSAL Python is the library that enables you to authenticate users and manage their access tokens.

After installing the required libraries, update your requirements file by running the following command:

Console

```
pip freeze > requirements.txt
```

Configure the application for authentication

Web applications that sign in users by using the Microsoft identity platform are configured through a configuration file, `.env`. In Python Flask, it must specify the following values:

Workforce tenant

 Expand table

Environment variable	Description
AUTHORITY	The URL of the cloud instance where the application is registered. Format: <code>https://{Instance}/{TenantId}</code> . Use one of the following Instance values: - <code>https://login.microsoftonline.com/</code> (Azure public cloud) - <code>https://login.microsoftonline.us/</code> (Azure US government) - <code>https://login.microsoftonline.de/</code> (Microsoft Entra Germany) - <code>https://login.partner.microsoftonline.cn/</code> (Microsoft Entra China, operated by 21Vianet)
TENANT_ID	The identifier of the tenant where the app is registered. Prefer the tenant ID from the app registration, or use one of: - <code>organizations</code>: sign in users in any work or school account - <code>common</code>: sign in users with any work or school account or Microsoft personal account - <code>consumers</code>: sign in users with a Microsoft personal account only
CLIENT_ID	Identifier of the application (client) as obtained from the app registration.
CLIENT_SECRET	The secret value obtained from add credentials in the Microsoft Entra admin center.
REDIRECT_URI	The URI where the Microsoft identity platform sends security tokens after authentication.

Update the configuration file

1. Create a `.env` file in your root folder to safely store your app's configuration. Your `.env` file should contain the following environment variables:

Workforce tenant

`.env`

```
CLIENT_ID="<Enter_your_client_id>"
CLIENT_SECRET="<Enter_your_client_secret>"
AUTHORITY="https://login.microsoftonline.com/<Enter_tenant_id>"
REDIRECT_URI="<Enter_redirect_uri>"
```

Replace the placeholders with the following values:

- Replace `<Enter_your_client_id>` with the *Application (client) ID* of the client web app that you registered.
- Replace `<Enter_tenant_id>` with the *Directory (Tenant) ID* where you registered your web app.
- Replace `<Enter_your_client_secret>` with the *Client secret* value for the web app you created. In this tutorial, we use secrets for demonstration purposes. In production, use more secure approaches such as [certificates or federated identity credentials](#).
- Replace `<Enter_redirect_uri>` with the redirect URI that you registered earlier. This tutorial sets the redirect URI path to `http://localhost:3000/getAToken`.

2. Create an `app_config.py` file to read the environment variables and add other configs that you need.

Python

```
import os

AUTHORITY = os.getenv("AUTHORITY")
CLIENT_ID = os.getenv("CLIENT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
REDIRECT_URI = os.getenv("REDIRECT_URI")
SESSION_TYPE = "filesystem" # Tells the Flask-session extension to store ses-
sions in the filesystem. Don't use in production apps.
```

Configure app endpoints

At this stage, you create your web app endpoints and add the business logic to your application.

1. Create a file called `app.py` in your root folder.
2. Import required dependencies at the top of the `app.py` file.

Python

```
import os
import requests
from flask import Flask, render_template
from identity.flask import Auth
import app_config
```

3. Initialize your Flask app and configure it to use the session storage type you specified in your *app_config.py* file.

Python

```
app = Flask(__name__)
app.config.from_object(app_config)
```

4. Initialize client the app client. A Flask web app is a confidential client. We pass the client secret because confidential clients can safely store it. Under the hood, the identity library calls the `ConfidentialClientApplication` class of the MSAL library.

Python

```
auth = Auth(
    app,
    authority=app.config["AUTHORITY"],
    client_id=app.config["CLIENT_ID"],
    client_credential=app.config["CLIENT_SECRET"],
    redirect_uri=app.config["REDIRECT_URI"]
)
```

5. Add the required endpoints to your Flask app. The web app uses the authorization code flow to sign in the user. The *ms_identity_python* MSAL wrapper library helps with interacting with the MSAL library hence making it easier to add sign in and sign out to your app. We add an index page and protect it using the `login_required` decorator provided by the *ms_identity_python* library. The `login_required` decorator ensures that only authenticated users can access the index page.

Python

```
@app.route("/")
@auth.login_required
def index(*, context):
    return render_template(
        'index.html',
        user=context['user'],
        title="Flask Web App Sample",
    )
```

User is guaranteed to be present because we decorated this view with `@login_required`.

Create the app templates

Create a folder called *templates* in your root folder. In the templates folder, create a file called *index.html*. This is the app's homepage. Add the following code to the *index.html* file:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{{ title }}</title>
</head>
<body>
  <h1>{{ title }}</h1>
  <h2>Welcome {{ user.get("name") }}!</h2>

  <ul>
    {% if api_endpoint %}
      <!-- If an API endpoint is declared and scopes defined, this link will show.
We set this in the call an API tutorial. For this tutorial, we do not define this
endpoint. -->
      <li><a href='/call_api'>Call an API</a></li>
    {% endif %}

    <li><a href="{{ url_for('identity.logout') }}">Logout</a></li>
  </ul>

  <hr>
  <footer style="text-align: right">{{ title }}</footer>
</body>
</html>
```

Run and test the sample web app

Workforce tenant

1. In your terminal, run the following command:

Console

```
python3 -m flask run --debug --host=localhost --port=3000
```

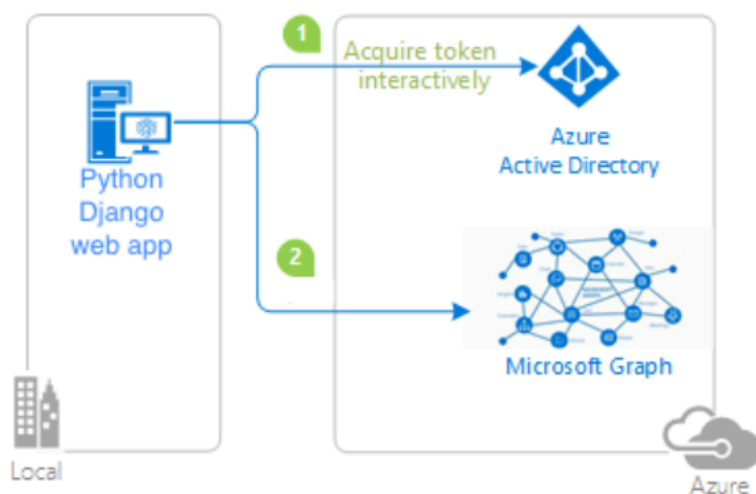
You can use the port of your choice. This port should be similar to the port of the redirect URI you registered earlier.

2. Open your browser, then go to `http://localhost:3000`. You see a sign-in page.
3. Sign in with your Microsoft account by following the steps. You're requested to provide an email address and password to sign in.
4. If there are any scopes needed by the application, a consent screen is presented. The application requests permission to maintain access to data you allow access to and to sign you in. Select **Accept**. This screen doesn't appear if no scopes are defined.

After you sign in or sign up, you're redirected back to the web app. You see a page that looks similar to the following screenshot:

Flask Web App Sample v0.9.0

Welcome Luus Van Hoogstraten!



- [Logout](#)

Flask Web App Sample v0.9.0

Select **Logout** to sign out of the app. You're prompted to pick an account to sign out from. Select the account you used to sign in.

Use custom URL domain (Optional)

Workforce tenant

Workforce tenants don't support custom URL domains.

Last updated on 12/01/2025